1a. Bulk upload (a zip of cases)

Code: `backend/app/api/bulk_uploads.py` (endpoints), `backend/app/services/bulk_upload_service.py` (zip rules, plan, ingestion), `backend/app/tasks/bulk_upload_task.py` (Celery task), `backend/app/services/document_intake.py` (the store → record → queue path shared with single uploads). **Screens:** `/cases/bulk` (upload) and `/bulk-uploads/:id` (live summary).

Many cases in one go: one zip file, **one top-level folder per case**, with that case's documents directly inside the folder. The zip is only a way to get cases in. Once its cases are created and their documents queued, its job is done. Every document then goes through exactly the same per-document pipeline as a single upload (01 onwards), with the same fair-share priority. Nothing in the pipeline reports back to the zip or waits for it.

```
claims.zip
├── case-001/   invoice.pdf  receipt.pdf
├── case-002/   فاتورة.pdf
└── case-003/   quotation.pdf
```

Limits

Limit	Default	Setting
Zip size	**per company**: `companies.max_zip_size_mb` (300 MB for a new company)	Platform › Companies › Edit; new-company default `DEFAULT_MAX_ZIP_SIZE_MB`
Each document	**per company**: `companies.max_file_size_mb` (10 MB for a new company), PDF only, plus every single-upload check	Platform › Companies › Edit; new-company default `DEFAULT_MAX_FILE_SIZE_MB`
Zip entries (files + folders)	25,000, a safety cap against zips of millions of tiny entries (a 300 MB zip of small born-digital PDFs holds about 11,000)	`BULK_UPLOAD_MAX_ENTRIES`
Cases per zip	**no hard cap**; a soft warning above 100	`BULK_UPLOAD_CASE_WARNING_THRESHOLD`

Both size limits are read from the uploading company's row (`services/upload_limits.py`) by the upload request and again by the ingestion task, the same source the single-file upload uses, and every rejection names the limit in effect for that company. The frontend shows the company's limits (from `GET /auth/me/upload-limits`), checks them before sending, and counts the case folders from the zip's central directory to show the warning.

Two stages

1. The upload request (`POST /bulk-uploads`, 202) returns as soon as the zip is stored.

- The body is the zip itself (`Content-Type: application/zip`), not multipart. A zip over the limit is refused **from its `Content-Length` before any of the body is read**. A body without a length is cut off as soon as it passes the limit.
- The body is streamed to a temporary file (it spills to disk above 8 MB) and hashed on the way.
- Only the **central directory** is read. No document is decompressed. Zip-level rejections, each with `detail: {code, message}`:

Code	HTTP	When
`zip_empty`	400	0 bytes

Code	HTTP	When
`zip_too_large`	413	over the company's zip limit (states the actual size and that limit)
`not_a_zip`	415	the header bytes are not a zip
`zip_corrupted`	422	the central directory can't be read (truncated or damaged)
`zip_too_many_entries`	422	over `BULK_UPLOAD_MAX_ENTRIES`
`zip_no_case_folders`	422	no folder holds any file

- It builds the **plan**: one `bulk_upload_cases` row per case folder. Every failure that needs no decompression is already marked, so the uploader sees it in the response, before anything is processed: a subfolder inside a case folder, an empty case folder, and 0-byte, oversized or zip-password-protected files.
- It stores the zip (`companies/{company_id}/bulk-uploads/{id}_{sha256}.zip`), writes the `bulk_uploads` row and its plan rows plus a `bulk_upload_received` audit event, counts the zip's storage, and queues **one** `ingest_bulk_upload` task.

2. Ingestion (`ingest_bulk_upload` , on `forensics_queue` : local CPU, no Azure call) goes case by case, in natural folder order (`case-2` before `case-10`):

1. Extract each document of the folder, never reading more than the company's per-file limit + 1 byte whatever the zip claims (zip-bomb safe). Then run the **same checks as a single upload** (`validate_upload` : size, PDF header bytes, extension, parse, password).
2. Store the accepted documents in Blob Storage like any single upload.
3. In **one short transaction**: create the case (submitted by the uploader, case type from the upload, `bulk_upload_id` , `reference_label` = folder name), its documents, their audit events and usage counters, and the folder's `bulk_upload_cases` row.
4. **Queue that case's documents immediately**, before the next folder is even extracted. The first case can be processed, scored and even reviewed while later cases are still being ingested.

No DB transaction is held across the zip download, extraction, validation or blob uploads.

Partial success

Situation	Result
One bad file in a case (corrupted, password-protected, too large, empty, not a PDF, wrong extension)	That file is rejected with the single-upload code and message. The rest of the case goes ahead
Every file in a case rejected	No case is created: `case_no_valid_documents`
A case folder contains a subfolder holding files	The whole folder is rejected (`case_nested_folder` , naming the subfolder). Files are never guessed into a case. An *empty* subfolder is ignored
An empty case folder	`case_no_documents`
Other case folders	Unaffected

Folder and file names

- **Non-ASCII names (Arabic etc.)** are read correctly whether the zip tool sets the UTF-8 flag, writes UTF-8 *without* the flag (common, and shown as mojibake by naive readers), or uses the Info-ZIP Unicode Path extra field. A legacy zip whose names are in a regional code page (e.g. CP720 from an old Windows "Send to → Compressed folder") can't be detected reliably and keeps its CP437 reading.
- **Folder names are labels, not identities.** Two folders that look alike (`Invoice` and `invoice`), or the same folder names in a later zip, become separate cases. Identity is always the generated case id / case number. Names are NFC-normalised, so the same name typed on macOS and on Windows is one folder.
- **One wrapper folder** around everything (right-click → Compress on a folder of case folders: `claims/case-001/…`) is detected and stripped once. The summary screen says so.
- **OS clutter** (`__MACOSX/` , `.DS_Store` , `Thumbs.db` , `desktop.ini` , dotfiles) is skipped. Loose files at the zip root belong to no case and are listed as ignored.

Summary screen

`GET /bulk-uploads/{id}` lists every case folder: created or failed (and why, per file) and, for created cases, a **live status** derived on every read from the case and its documents:

`validating` (not ingested yet) → `queued` (no document picked up by a worker) → `processing` → `done` , or `flagged` (medium/high risk tier). A submitter sees `done` instead of `flagged` and never sees the tier, the same rule as the case queue. The screen polls every 3 s until `settled` .

Visibility follows the case rules: a `user` sees only their own uploads, reviewers see their company's, and platform admins get read-only, audited access (`company_id` query parameter). Every company role may bulk upload; platform admins may not (403).

Resuming after a crash

Each case is committed together with its `bulk_upload_cases` row, so a redelivered task (the worker died) or a retried one (Blob Storage hiccup: up to 3 retries) skips folders already handled and never duplicates a case. If ingestion fails for good, the upload is marked `failed` with a message. Cases already created stay created and keep processing.

Sizing (measured)

`python -m scripts.bulk_upload_benchmark --profile scanned|digital` builds a zip close to 300 MB and times both stages (in-memory storage and SQLite, so this is the CPU floor):

Profile	Cases / documents in ~295 MB	Upload request (inspect)	Ingestion	First document queued
Scanned (2 × ~2 MB per case)	73 / 146	21 ms	3.9 s	0.8 s
Born-digital (3 × ~40 KB per case)	2,744 / 8,232	185 ms	132 s (≈ 48 ms per case)	1.4 s

A real 300 MB zip therefore holds anything from about 70 cases (scanned evidence) to about 3,000 (born-digital invoices). Unzipping and validating in the request would hold it for seconds to minutes, which is why ingestion is a background task.

Known limitations

- Every case in a zip gets the same case type.
- The zip is kept in Blob Storage as the record of the submission (and counted in the company's storage). There is no retention or cleanup job.
- A worker crash between storing a case's documents and committing the case leaves orphaned blobs, the same as a single upload that fails after storage.
- Ingestion of one zip is sequential on one `forensics_queue` worker. Very large born-digital zips take minutes to ingest fully, while their first cases are already processing.